שיעור 23

מתחילים...

Routers

Routers מאפשרים לנו לשנות את כתובת ה URL בהתאם לעמוד שבו אנו נמצאים.

כאשר עוברים בין עמודים באתר מתבצע מעבר בין ערוצים.
ניתן לראות שכתובת ה URL משתנה בהתאם לערוצים.

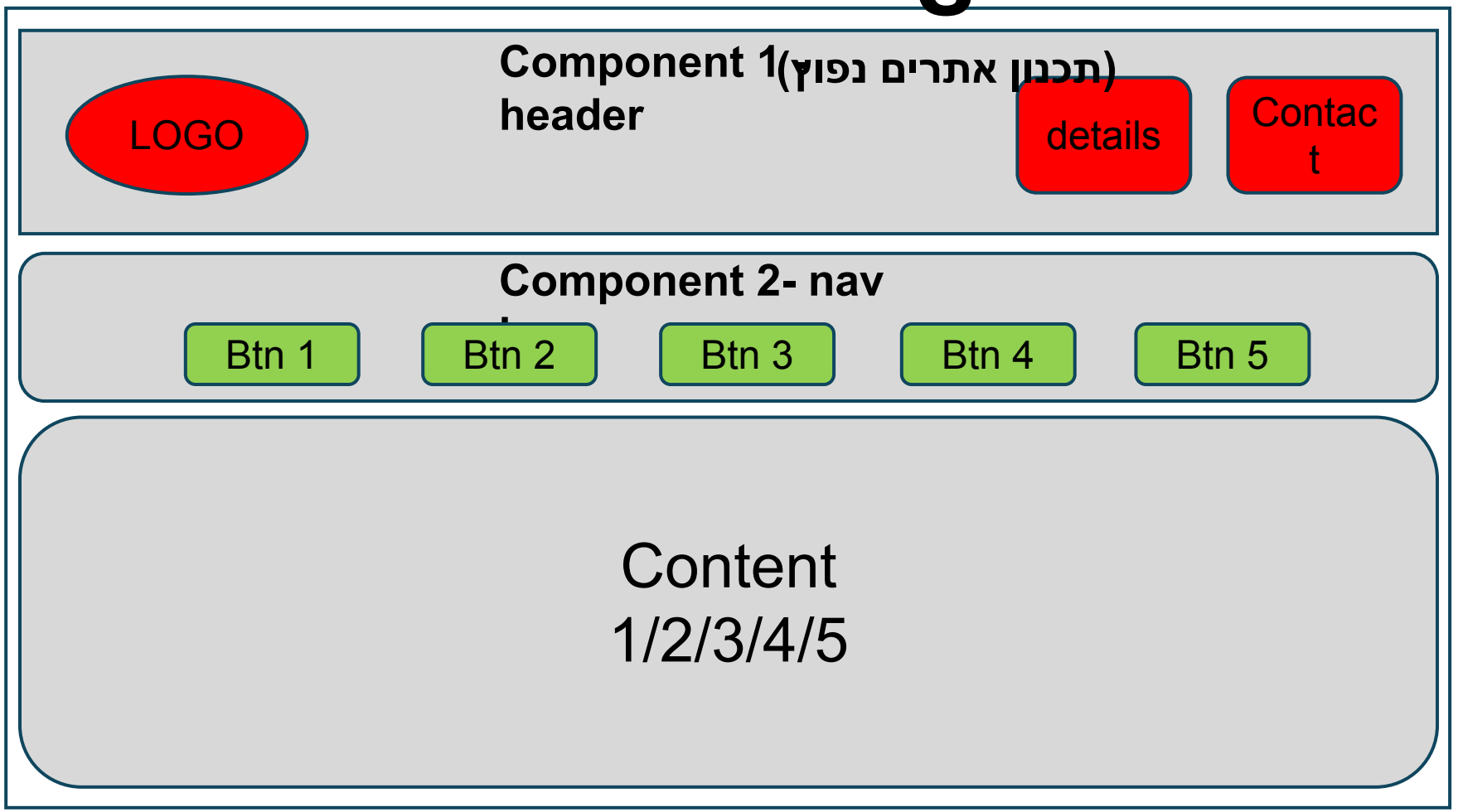
<u>דוגמא</u>

הערוץ הראשי של המכללה:
https://svcollege.co.il

ערוץ הקורסים של המכללה (לחיצה על "כל הקורסים"):
https://svcollege.co.il/courses/

ערוץ קורס פיתוח:https://svcollege.co.il/courses/web-development/

Routers



לא קשור לשינוי הערוצים

קשור לשינוי הערוצים אבל לא מתחלף

קשור לשינוי הערוצים וגם מתחלף בעצמו

Routers

כדי לעבוד עם ערוצים יש להוסיף את ה module למערכת שלנו (ב- CLI):

```
npm install react-router-dom
```

בקומפוננטה App:

```
import {BrowserRouter, Routes , Route} from 'react-router-dom';
```

Routers

< BrowserRouter /> – בתוכו יהיו הרכיבים הקשורים לערוץ (מתחלפים או לא מתחלפים).
<Routes/> – בתוכו יהיו הרכיבים הקשורים לערוץ שאמורים להתחלף בפועל.
<Route/> – הגדרת ערוץ עבור קומפוננטה ספציפית.

```
<h1>not related to the Routers</h1>

< BrowserRouter >
    <Routes>
        <Route path='/routhPath'  element={<ComponentName/> } />
    </Routes>
</ BrowserRouter >
```

**במידה ונגדיר: path='/' הערוץ יהיה דף הנחיתה (הראשי)

Routers - Link

ניצור קומפוננטה Menu שבה יהיו כפתורי הניווט.
נרצה שכל כפתור יעביר אותנו לערוץ הרלוונטי ולכן נוסיף ב Menu את הפקודה:

```
import {Link} from 'react-router-dom';
```

נרצה שבעת לחיצה על כפתורי הניווט יבוצע מעבר לערוץ הרצוי באתר.
נעשה זאת באמצעות הפקודה הבאה:

```
<Link to='/'> LinkName </Link>
```

לאיזה נתיב לבצע את ההחלפה

שם הלינק (הכפתור)

Menu צריך להיות בתוך BrowserRouter כדי להשתמש ביכולות הניווט

Routers - Link

JS App.js | JS Menu.js | JS HomePage.js | JS First.js | JS Secon

src > Components > JS Menu.js > ...

```
1   import React from 'react';
2   import { Link } from 'react-router-dom';
3
4   export default function Menu() {
5       return (
6           <div className='row' style={{border:'2px solid gray'}}>
7               <div className='col-4'>
8                   <Link to='/'>Home</Link>
9               </div>
10              <div className='col-4'>
11                  <Link to='/First'>First</Link>
12              </div>
13              <div className='col-4'>
14                  <Link to='/Second'>Second</Link>
15              </div>
16          </div>
17      )
18  }
```

ב Menu יצרנו לינק לכל ערוץ.
כאשר המשתמש ילחץ על אחד
הלינקים (Home, First או Second) הערוץ
ישתנה בהתאם וכך יחזיר את הקומפוננטה
שהוגדרה בו והיא תוצג על המסך.

Routers - useNavigate

הפיצ׳ר הבא שמספק לנו החבילה, הואuseNavigate
ההבדל בין מה שלמדנו מקודם למה שנלמד עכשיו, הוא שב Link
.אין לנו דרך להעביר נתיב לפני תנאי, כלומר זה תמיד יעבוד

לפעמים אנחנו רוצים להעביר את המשתמש עמוד אך ורק אם הוא נרשם בהצלחה, אם הוא עמד
.בכל התנאים, וכן הלאה
פה נכנס לתמונה
useNavigate

```
export default function SignIn({ setIsLoggedIn, users ,setLoggedInUserId}) {
    const [name, setName] = useState("");
    const [pass, setPass] = useState("");
    const nav = useNavigate();

    const checkPass = () => {
        const user = users.find(user => user.name === name && user.password === pass);
        if(user) {
            setIsLoggedIn(true);
            setLoggedInUserId(user.id)
            nav('/homepage')
        } else {
            alert('Wrong username or password');
        }
    };
```

הכנסת ההוק למשתנה

צורת השימוש

משימה Routers

1. צרו פרוייקט חדש בשם TEST ROUTERS עם 3 עמודים שכל אחד מכיל כותרת , עמוד בית, אודות וצור קשר
2. צרו תפריט משותף לכולם המאפשר מעבר בין עמודים
3. בונוס: עצבו את התפריט והוסיפו פריטים לעמודים . המטרה היא ליצור אתר למותג בגדים כלשהו

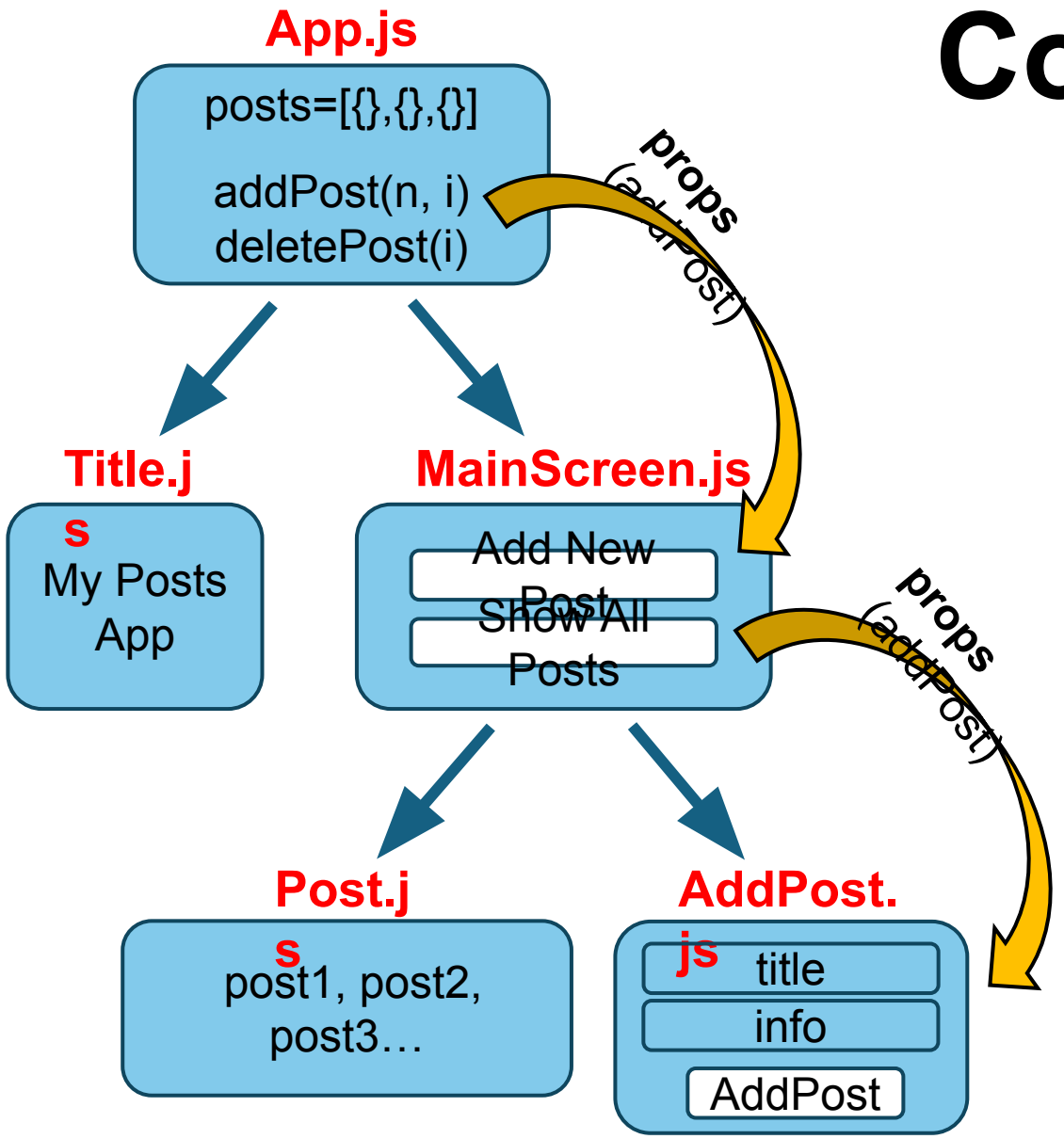
אז מה בעצם ראינו כאן?

יצרנו nav bar שכל כפתור ניווט יפנה לערוץ המתאים באמצעות Link.

יצרנו ערוצים ב App שיודעים להחזיר קומפוננטות.

תכננו את מבנה האתר: מה לא קשור לערוצים, מה קשור ולא משתנה ומה כן משתנה.

Context



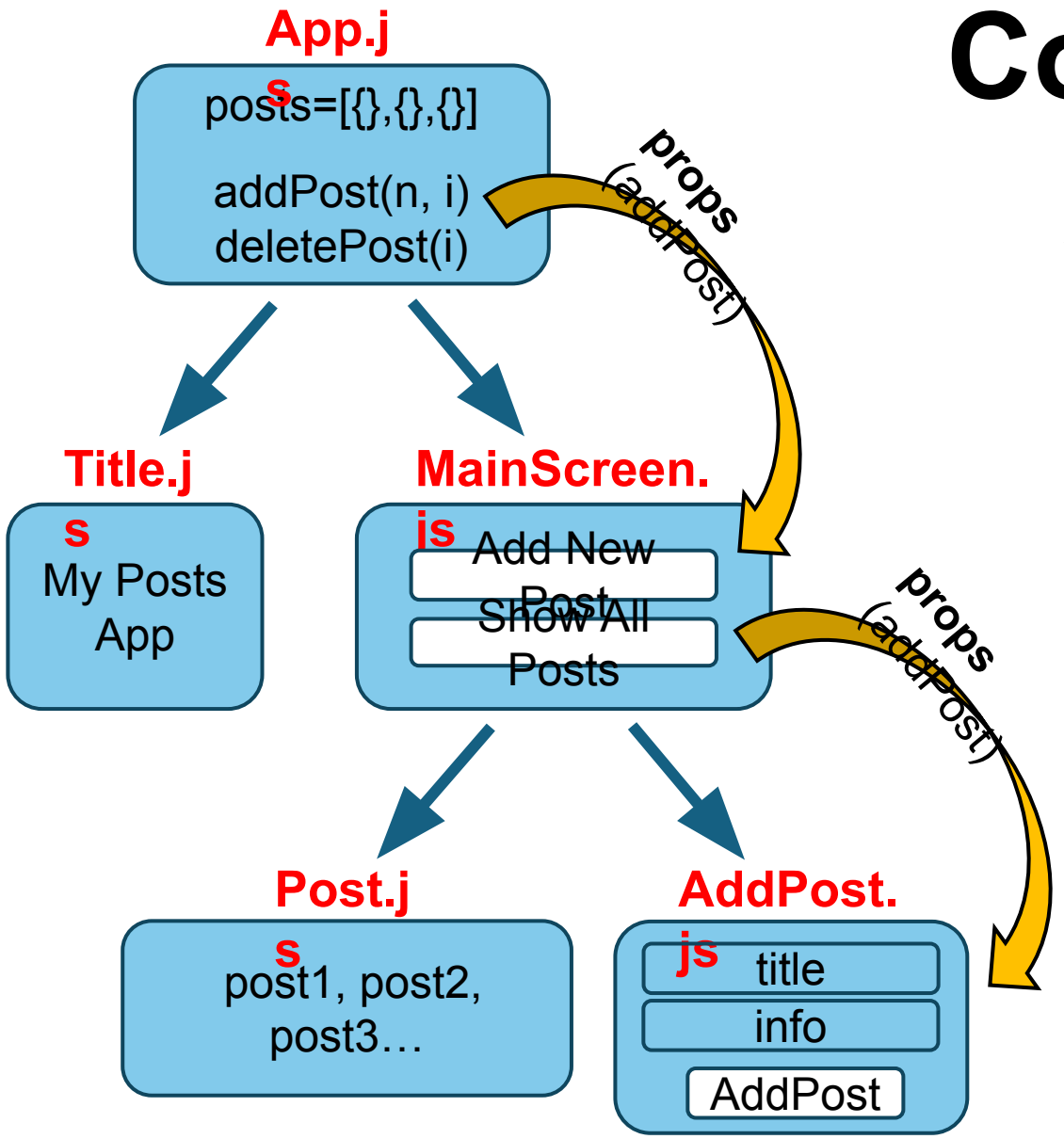
כאשר מעבירים props בין קומפוננטות הם יעברו גם דרך קומפוננטות ביניים שלעתים כלל לא ישתמשו במידע .

בדוגמא זו הפונקציה addPost נשלחת מ App ל AddPost באמצעות props ועוברת דרך MainScreen שלא באמת עושה בה שימוש .

באותו אופן זה קורה עם מערך הפוסטים והפונקציה deletePost שנשלחות מ App ל Post באמצעות props .

נרצה להעביר מידע מקומפוננטה אחת לאחרת ללא מעבר דרך קומפוננטות ביניים.
נעשה זאת באמצעות ContextAPI.

Context



דוגמא להעברת props (addPost) לפני שימוש ב ContextAPI:

App:

<MainScreen add={addPost} />

שליחת המאפיין add מ App ל MainScreen
(ערך המאפיין הוא הפונקציה addPost).

MainScreen:

<AddPost add={props.add}

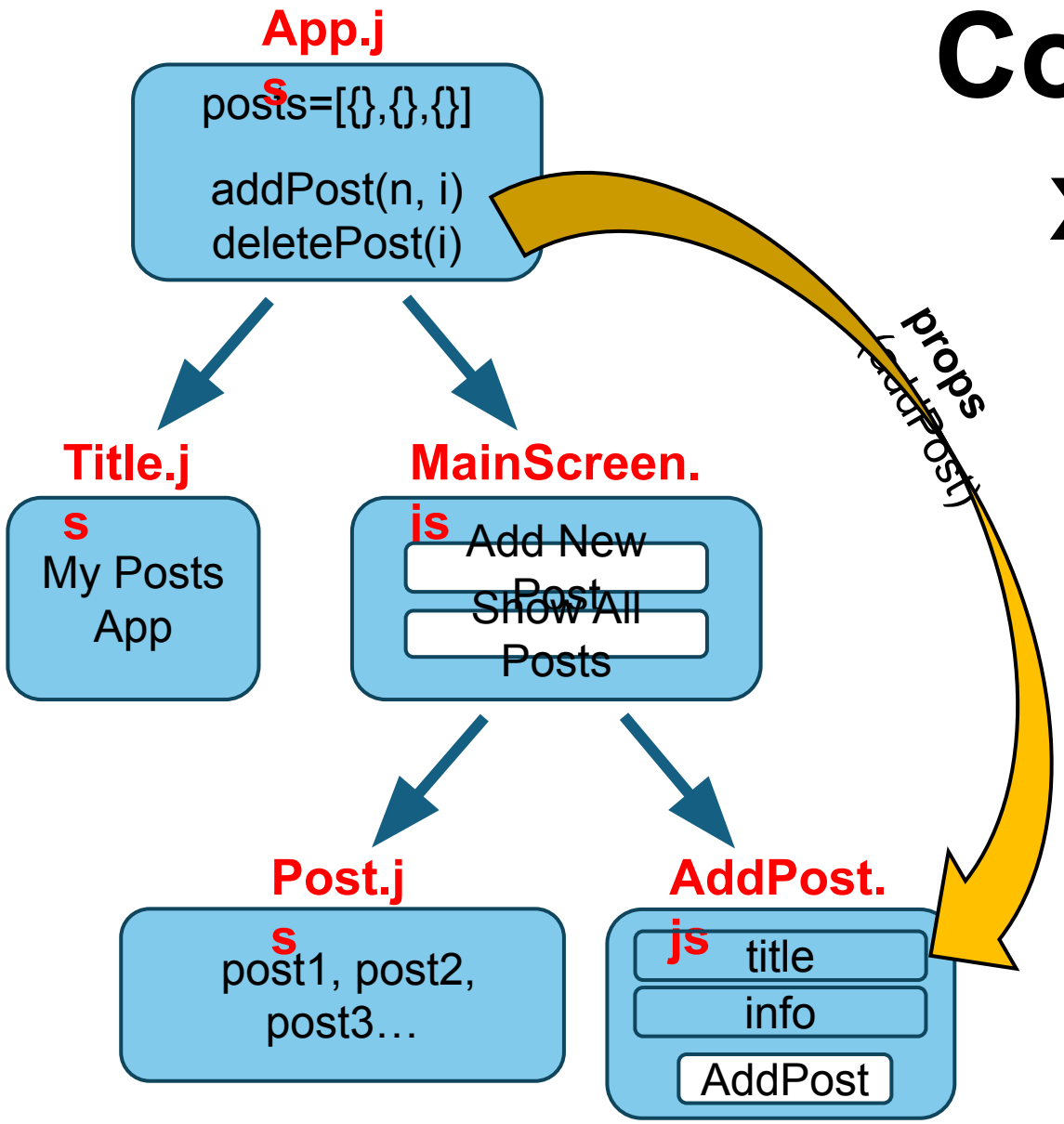
שליחת המאפיין Add מ MainScreen ל AddPost
(הערך הוא אותו add שנשלח באמצעות props מ App).

AddPost:

props.add(name, info);

שימוש במאפיין שנשלח מ App, שליחת ערכים לפונקציה.

Context



באמצעות ContextAPI נוכל לדלג על השרשרת ולהעביר props באופן ישיר מקומפוננטה אחת לאחרת.

Conte
xt

ניצור קומפוננטה ContextAPI.js ונכתוב בה:

```
Import {createContext} from ”react”

export const AppContext = createContext();
```

Context

נגדיר את App שיהיה ה Provider:

import {AppContext} from ‘./context'

נעטוף את MainScreen שנמצא ב App ב Provider (היות והוא מספק את כל ה props).

```
<Provider value={{addPost}}> // הגדרנו אובייקט לאפשר שליחה של כמה מאפיינים ביחד
    <MainScreen ~~add={addPost}~~ />
</Provider>
```

נמחק את המאפיין add ששלחנו, היות ודאגנו לשלוח אותו באמצעות Provider.

מהקומפוננטה MainScreen נמחק את המאפיין add היות והקומפוננטה אינה תחנת מעבר.

```
<AddPost ~~add={props.add}~~ />
```

**value* הינה מילה שמורה.

useContext

לאחר שעטפנו את כל הקומפוננטות שלנו בContext
אנחנו רק צריכים למשוך את הנתונים שאנחנו רוצים, זמינים בכל קומפוננטה.

```
import React, { useContext } from 'react'
import { AppContext } from '../contextApi'

export default function AddPost() {
    const {addPost} = useContext(AppContext)
    //addPost received in child component

  return (
    <div>
        <button>I can add post now!</button>
    </div>
  )
}
```

נרצה לגשת לתוכן ששלחנו, ואנחנו נעשה את זה בעזרת ההוק useContext , אשר יודע למשוך את הנתונים שנשלחו על ידי הProvider.
ההוק useContext מקבל בפרמטר את משתנה הAppContext, ובכך יש לו גישה לכל הערכים שנשלחו.

במקרה שבתמונה עשינו פעולת Object Destructring
כי רצינו לקבל רק את addPost.

useParams

חבילת react-router-dom מספקת לנו גם הוק בשם useParams , אשר יודע לגשת לפרמטרים בurl.

לדוגמא אם אני רוצה לבנות נתיב דינאמי, שמספק למשתמש את פרטי הפרופיל שלו.

אני לא אצור נתיב חדש לכל משתמש שמתחבר, אני אצור נתיב דינאמי בצורה הבאה:

```
<Route path="/user/:userId" element={<User users={users}/>} />
```

שימו לב ל":" אחרי הלוכסן, ה: מסמן את הפרמטר שהוא הuserId.

נצא מנקודת הנחה שאלו רשימת המשתמשים שלנו:

```
const [users, setUsers] = useState([
  { id: '1', name: 'Alice', password: 'alice123' },
  { id: '2', name: 'Bob', password: 'bob123' },
  { id: '3', name: 'Charlie', password: 'charlie123' },
]);
```

usePara
ms

שימו לב שהפעם אנו מעבירים לנתיב דינמי ישתנה בהתאם למשתמש המחובר!

```
const checkPass = () => {
    const user = users.find(user => user.name === name && user.password === pass);
    if(user) {
        setIsLoggedIn(true);
        setLoggedInUserId(user.id)
        nav(`/user/${user.id}`);

    } else {
        alert('Wrong username or password');
    }
};
```

עכשיו הנתיב ישתנה בהתאם ליוזר שהתחבר, במידה והתחבר bob , יופיע user/2/
במידה והתחבר Charlie, יופיע user/3/

וכך הלאה.

usePara
ms

אנחנו גם יכולים לעשות שימוש בפרמטרים האלו בעזרת useParams

```
export default function User({ users }) {
  const { userId } = useParams();
  const user = users.find(user => user.id === userId);

  return (
    <div>
      <h2>User Profile</h2>
      {/* Display the username if user is found */}
      {user ? <p>Username: {user.name}</p> : <p>User not found</p>}
    </div>
  );
```

סוף שיעור 23

האם יש שאלות?